



Islington college
(इसलिंग्टन कलेज)

Module Code & Module Title

Level 7 – Project

Dissertation Topic: Design and Evaluation of an Agentic Retrieval-Augmented Generation Framework for Real-Time System Log Analysis and Alerting

Assessment Type

Semester

2025 Autumn

Student Name: Sharad Chandra Paudel

London Met ID: 23057169

College ID: NP01MS7S240007

Assignment Due Date: Friday, January 16, 2026

Assignment Submission Date: Thursday, January 15, 2026

Supervisor Name: Aadesh Tandukar

Word Count (Where Required): 3088

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

1. Introduction

Distributed systems today produce a large amount of log data that record system operations and potential failures. Research indicates that these systems can generate terabytes of log data daily (Chen et al. 2021). While logs offer useful information for maintaining system reliability, their volume and complexity make manual analysis impossible (He et al. 2020).

Traditional log analysis methods have gone through three main stages. First, rule-based systems used manually written patterns to identify known issues. These systems were predictable, but they were also inflexible and produced many false alerts (Du et al. 2017). Second, machine learning methods like Principal Component Analysis and Isolation Forest improved detection by learning from data, but they couldn't explain their findings (Xu et al. 2009). Third, deep learning methods such as DeepLog and LogBERT achieved better accuracy (Du et al. 2017; Guo et al. 2021) but still lacked integration with external knowledge and offered limited explanations.

Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) present new opportunities. RAG combines LLMs with retrieval from external knowledge sources, which enhances accuracy and reduces errors (Lewis et al. 2020; Gao et al. 2023). Agentic AI systems that employ multi-step reasoning have proved effective for complex tasks (Yao et al. 2023). However, no research has examined how well agentic RAG performs in log analysis compared to existing methods.

This dissertation will design, implement, and evaluate an agentic RAG framework for log analysis. The framework will use multi-step reasoning over retrieved knowledge to create understandable alerts. Through experiments that compare it against three baseline approaches, this research will provide evidence on whether agentic RAG improves log analysis.

2. Problem Statement

There is a constant challenge for system administrators and cybersecurity professionals due to the growing volumes of logs. Modern systems produce continuous log streams that overwhelm human analysis capacity (He et al. 2020). When log analysis fails, the consequences can include service outages, security issues, longer response times, and potential financial loss or damage to reputation.

Current automated log analysis methods face three main problems:

1. Rule-based systems are inflexible. Manually created patterns do not adapt to new types of failures, leading to too many false alerts or missed real problems (Du et al. 2017).
2. Machine learning methods can improve detection, but they operate like black boxes. They provide scores without explaining why an alert was raised, making it hard for operators to understand the causes (Le and Zhang 2022).
3. These methods do not utilize external knowledge, which can lead to incorrect interpretations, especially for unclear or new log patterns (Ji et al. 2023).

LLMs can offer better understanding, but they lack specific operational knowledge, and their training data can become outdated (Gao et al. 2023). LLMs also often create believable but incorrect information, especially in technical areas (Ji et al. 2023).

RAG tackles these issues by combining LLM generation with information retrieval from external knowledge sources (Lewis et al. 2020). However, most RAG systems rely on single-step retrieval, which may not be enough for complex log interpretation. Agentic AI systems use multi-step reasoning loops for more advanced problem-solving (Yao et al. 2023).

3. Dissertation Aim and Objectives

3.1 Aim

The primary aim of this dissertation is to design, implement, and evaluate an agentic RAG framework for system log analysis that uses multi-step reasoning and knowledge retrieval to improve alert accuracy and interpretability compared to existing approaches.

3.2 Objectives

- **O1:** Review existing research on log analysis techniques, RAG systems, and agentic AI to find gaps and build foundations.
- **O2:** Design an agentic RAG architecture that includes log preprocessing, multi-step reasoning, knowledge retrieval, and alert generation components.
- **O3:** Implement the framework using open-source tools like Python, LangChain, FAISS, and Llama2/Mistral to create a working prototype.

- **O4:** Set up baseline systems for comparison, including rule-based analysis, classical machine learning (Isolation Forest), and standard RAG (non-agentic).
- **O5:** Evaluate all systems on public log datasets (HDFS, BGL) using metrics that measure detection accuracy (precision, recall, F1-score) and computational efficiency (latency, resource usage).
- **O6:** Analyze the results to see when agentic RAG performs better or worse than the baselines and document any limitations.

4. Motivational Background

Log analysis is important for both operational and economic reasons. Research shows that unplanned downtime costs businesses about \$5,600 per minute (Gartner 2023). System logs allow for early problem detection that can prevent failures, but current methods have significant drawbacks.

From a research standpoint, log analysis is suitable for evaluating agentic RAG because it requires domain knowledge, reasoning over time, and understanding meaning. This makes it a good setting to test whether multi-step reasoning with retrieval is better than simpler methods.

This research fills a gap in the RAG literature. While some studies indicate that RAG works well for question-answering (Gao et al. 2023), operational monitoring has not been examined. Recent work in agentic AI has focused on web navigation and code generation (Yao et al. 2023), leaving unanswered questions about effectiveness in time-sensitive operational settings.

Log analysis provides several research benefits. First, public benchmark datasets (HDFS, BGL) with labels allow for objective measurement (Xu et al. 2009; Oliner and Stearley 2007). Second, this field presents realistic challenges, including noisy data and changing patterns. Third, findings directly relate to the needs of practitioners.

Improving the reliability of log analysis can lead to better system monitoring, which enhances infrastructure stability and prevents disruptions in healthcare, finance, and critical infrastructure. However, the research will also look at the risks of over-automation.

5. Research Questions

This dissertation focuses on two primary research questions:

RQ1: As compared to non-agentic RAG & rule-based log analysis approach, does agentic RAG improve detection accuracy?

RQ2: Compared to baseline approaches, what are the computational costs of multimodal agentic AI and RAG?

6. Literature Review

6.1 Log Analysis Techniques

There has been advancement in log analysis across the time and technology advancement. Early rule-based systems relied on manually specified patterns for known failures (He et al. 2017). While these methods were predictable, they lacked flexibility. New failures went undetected, and maintaining the patterns became impractical.

Machine learning brought data-driven detection. Xu et al. (2009) pioneered **Principal Component Analysis (PCA)** -based methods for **Hadoop Distributed File System** log analysis, identifying statistical outliers. Liu et al. (2008) introduced Isolation Forest for unsupervised anomaly detection. However, these methods relied on hand-crafted features and could not capture semantic meaning.

On other hand, Deep learning methods learned representations directly from data. Du et al. (2017) proposed DeepLog, which used **Long Short-Term Memory (LSTM)** networks to model log sequences and predict expected events. When evaluated on HDFS datasets, DeepLog achieved F1-scores above 0.95. Meng et al. (2019) built on this with LogAnomaly, which added semantic embeddings.

Guo et al. (2021) introduced LogBERT, applying transformer models to log analysis through self-supervised pre-training. LogBERT achieved F1-scores of 0.982 on HDFS and 0.908 on BGL datasets. However, Le and Zhang (2022) found that performance declined under realistic conditions, including noisy labels and changing formats. All neural methods provide numerical scores but lack human-understandable explanations.

The identified gaps to this approach was that even though Deep learning methods achieve high accuracy but they still lacked external knowledge integration and interpretability.

6.2 Retrieval-Augmented Generation (RAG)

With the advancement of AI models, RAG emerged as a solution to the knowledge limits of LLMs. Lewis et al. (2020) introduced the basic structure that combines dense retrieval with generation. They showed that retrieved passages greatly improve performance on knowledge-heavy tasks. The main idea is that parametric knowledge in model weights works alongside non-parametric knowledge retrieved from external sources.

Gao et al. (2023) offered a very detailed classification of RAG methods. Basic RAG models use a simple retrieve-then-generate approach but it still faces issues related to the accuracy on the retrieval. Alternatively, Advanced RAG adds pre-retrieval optimization and post-retrieval refining. Modular RAG supports iterative retrieval and flexible strategies. Their study indicates that RAG consistently lowers errors compared to separate LLMs.

The recent studies have looked into domain-specific RAG. Zhang et al. (2024) showed that RAG-enhanced clinical decision support reached greater accuracy when using medical records. Blair et al. (2023) found that RAG is effective for legal analysis, as grounding in statutory text reduced incorrect citations.

The existing gap in this approach is that the RAG has been studied for simple question-answering approach but not for operational monitoring domains with continuous data and time-sensitive requirements. Most RAG research evaluates single-step architectures.

6.3 Agentic AI and Multi-Step Reasoning

Agentic AI systems feature autonomous decision-making and iterative improvement. Yao et al. (2023) introduced ReAct (Reasoning and Acting), which allows large language models (LLMs) to produce interleaved thought processes and actions. In benchmarks that require multi-hop reasoning, ReAct outperformed standard

prompting by 34%. This shows that combining action-taking with reasoning helps reduce errors.

Schick et al. (2023) introduced Toolformer, which teaches models when to use external tools. LangChain made composable architectures popular for creating agents powered by LLMs.

However, agentic systems come with challenges. Longer inference chains raise computational costs because each reasoning step needs separate processing. There is also a risk of error propagation; incorrect reasoning in earlier steps can affect later actions. Evaluating performance becomes more complex with multi-step trajectories.

The gap in this approach is despite having the potential for complex tasks, agentic AI has been limitedly explored in time-sensitive operational contexts like log analysis. There has been no systematic comparison that measures when multi-step reasoning is worth the computational costs.

6.4 Evaluation Frameworks

In contrast to anomaly detection during log analysis, He et al. (2020) introduced Loghub, which includes 19 log datasets. HDFS (Xu et al. 2009) has 11 million log entries with block-level labels. BGL (Oliner and Stearley 2007) consists of 4.7 million entries from supercomputers. These datasets allow for standardized evaluation, but Le and Zhang (2022) warn that label noise and sampling strategies can affect performance.

Standard metrics include precision, recall, F1-score, false positive rate (FPR), and false negative rate (FNR). However, these metrics do not address interpretability. Chen et al. (2023) suggested human evaluation protocols that check if explanations are complete and actionable.

For RAG systems, Es et al. (2023) introduced RAGAS, which evaluates faithfulness (how well the output is based on context), relevance, and retrieval quality.

The gap in the frameworks is that integrated method that assesses both detection performance and interpretability for RAG-based log analysis was not found.

6.5 Summary

Advancement of Log analysis follows the trajectory, all the way from rule-based systems to machine learning, and now it includes transformer-based methods. These advancements have improved accuracy but still struggle with interpretability and knowledge grounding. RAG proves effective by basing generation on retrieved evidence. Agentic AI offers potential for complex reasoning. However, no research has designed and assessed an agentic RAG framework specifically for log analysis, with empirical comparison against established methods. This aim of this dissertation is to fill the existing gap to deliver a system using multimodal Agentic AI.

7. Methodology Selection

7.1 Research Methodology (Quantitative)

The methodology selected is the Comparative experimental design. The reason to select Comparative experimental design is that the performance detection for Log anomaly can be conveniently measured using quantitative metrics like precision, recall, and F1-score on datasets with ground-truth labels. The research question, "does agentic RAG outperform baselines?" requires controlled experiments that isolate variables.

The selected methodology can involve the implementation of multiple systems (agentic RAG, non-agentic RAG, classical ML, rule-based). It can also evaluate these multiple system on identical datasets. Comparative experimental design also enables direct performance comparison and at the same time evaluation of controlling factors like dataset characteristics and implementation quality. Statistical significance testing (t-tests) will validate whether observed differences exceed random variation.

The noted variables for the analysis are mentioned below:

- **Independent Variable:** Log analysis approach within existing systems
- **Dependent Variables:** Precision, Recall, F1-score, Latency, Memory consumption
- **Control Variables:** Same datasets, same train/test splits, comparable implementation effort

The proposed datasets to be used during the evaluation would primarily be HDFS and BGL public log datasets with ground-truth labels, providing thousands of sequences for statistically valid results.

7.2 Development Methodology

The proposed framework for the development is “Waterfall Software Development Lifecycle Model”. The Waterfall model is chosen for this dissertation because it offers a clear, step-by-step method that fits well with academic research. This model works well with specific goals and set timelines. Unlike commercial software projects that need ongoing user feedback, this research has clear requirements: specific research questions, set evaluation metrics, and public benchmark datasets (HDFS, BGL). The linear flow of Waterfall includes the requirements, design, implementation, testing, evaluation, documentation. This fits smoothly with the stages of academic research, where each phase relies on the work done before it. This structure guarantees thorough documentation at every stage, which is important for meeting dissertation standards and ensuring that results can be repeated in future studies. Additionally, the proposed timeline and the involvement of a single developer make Waterfall's clear milestones crucial for tracking progress effectively and finishing on time.

7.2.1 Waterfall Phases:

Phase	Key Activities
Phase 1: Requirements Analysis	• Complete literature review and identify system requirements. • Define functional specifications for agentic RAG framework. • Establish evaluation criteria and metrics • Acquire and preprocess HDFS and BGL datasets. • Configure development environment (Python 3.10+, FAISS, LangChain, Ollama)
Phase 2: System Design	• Design agentic RAG architecture with all components • Specify interfaces for Log Preprocessor, Agentic Controller, Retrieval System, LLM Engine, and Alert Generator

		• Design knowledge base schema and vector indexing strategy • Design ReAct-inspired multi-step reasoning workflow • Specify baseline system architectures (Rule-based, Isolation Forest, Non-agentic RAG)
Phase 3: Implementation		• Implement Log Preprocessor using Drain3 parser • Implement Rule-based baseline with regex patterns • Implement Isolation Forest baseline with TF-IDF vectorization • Build and populate knowledge base; create FAISS vector index • Implement Non-agentic RAG baseline • Implement Agentic RAG framework with multi-step reasoning controller • Conduct integration testing
Phase 4: Testing and Validation		• Conduct unit testing for individual components. • Perform integration testing for all four systems • Validate preprocessing correctness and retrieval quality • Execute pilot experiments to verify evaluation pipeline
Phase 5: Evaluation		• Execute comparative experiments on HDFS and BGL datasets for all systems • Collect detection metrics (Precision, Recall, F1-score, FPR) • Collect efficiency metrics (latency, memory consumption) • Perform statistical significance testing (paired t-tests) • Analyze and interpret results
Phase 6: Documentation		• Analyze results and answer research questions • Write dissertation chapters (Introduction, Literature Review, Methodology, Implementation, Results, Discussion, Conclusion) • Revise and proofread complete document

- | | |
|--|--|
| | <ul style="list-style-type: none">• Format according to university guidelines and submit |
|--|--|

7.2.2 System Architecture:

The proposed framework for the Agentic RAG will consists of:

- **Log Preprocessor:** Parses and normalises log entries using Drain3
- **Agentic Controller:** Implements multi-step reasoning (ReAct-style with max 3-5 steps)
- **Retrieval System:** FAISS vector database with historical incidents and documentation
- **LLM Engine:** Mistral-7B for generation via Ollama
- **Alert Generator:** Produces structured alerts with explanations

Baseline Systems:

- **Rule-Based:** Regular expressions for ERROR/FATAL keywords with threshold triggers
- **Classical ML:** Isolation Forest on TF-IDF log representations
- **Non-Agentic RAG:** Single retrieve-then-generate cycle (k=5 retrievals)
- **Agentic RAG:** Multi-step reasoning with iterative retrieval (proposed system)

Technology Stack (Open-Source):

- Language: Python 3.12
- Log Parsing: Drain3
- Vector Store: FAISS
- LLM: Mistral 7B Instruct
- Framework: LangChain
- ML Baseline: Scikit-learn
- Evaluation: Jupyter Notebooks, Matplotlib

Evaluation Metrics:

- **Detection Performance:** Precision, Recall, F1-score, False Positive Rate
- **Efficiency:** Average latency per log analysis, memory usage
- **Statistical Analysis:** Paired t-tests for significance testing.

Agentic RAG System Architecture for Log Analysis

System Architecture Flow

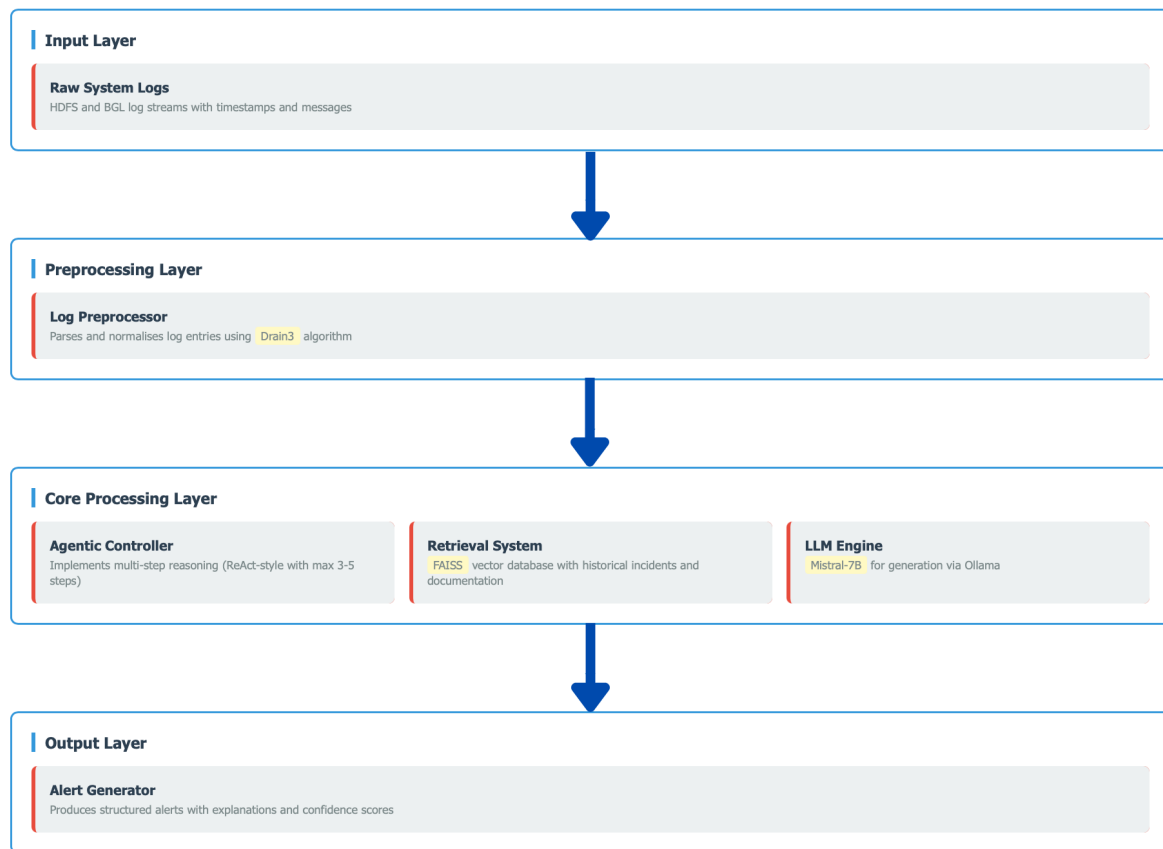


Fig 1: System Architecture Design

8. Project Plan

8.1 Timeline Overview

The dissertation spans 11 weeks from February 16 to May 5, 2026, following the Waterfall Software Development Lifecycle:

Phase	Duration	Timeline	Key Activities
Phase 1: Requirements Analysis	1.5 weeks	Feb 16 - Feb 25, 2026	- Complete literature review. - Define system requirements. - Acquire and preprocess HDFS and BGL datasets. - Setup development environment

Phase 2: System Design	1.5 weeks	Feb 26 - Mar 07, 2026	• Design agentic RAG architecture. • Specify component interfaces. • Design knowledge base schema. • Design baseline systems
Phase 3: Implementation	3.5 weeks	Mar 08 – Mar 31, 2026	• Implement Log Preprocessor • Implement all baseline systems (Rule-based, Isolation Forest, Non-agentic RAG) • Develop agentic RAG framework • Conduct integration testing
Phase 4: Testing and Validation	1.5 weeks	Apr 01 - Apr 10, 2026	• Unit testing. • Integration testing. • Validate preprocessing and retrieval quality. • Execute pilot experiments
Phase 5: Evaluation	1.5 weeks	Apr 11 - Apr 20, 2026	• Run comparative experiments on HDFS and BGL datasets. • Collect detection and efficiency metrics. • Perform statistical analysis. • Interpret results.
Phase 6: Documentation	2.5 weeks	Apr 21 - May 5, 2026	• Analyse result. • Write all dissertation chapters. • Revise and proofread. • Format and submit final document

8.2 Gantt Chart

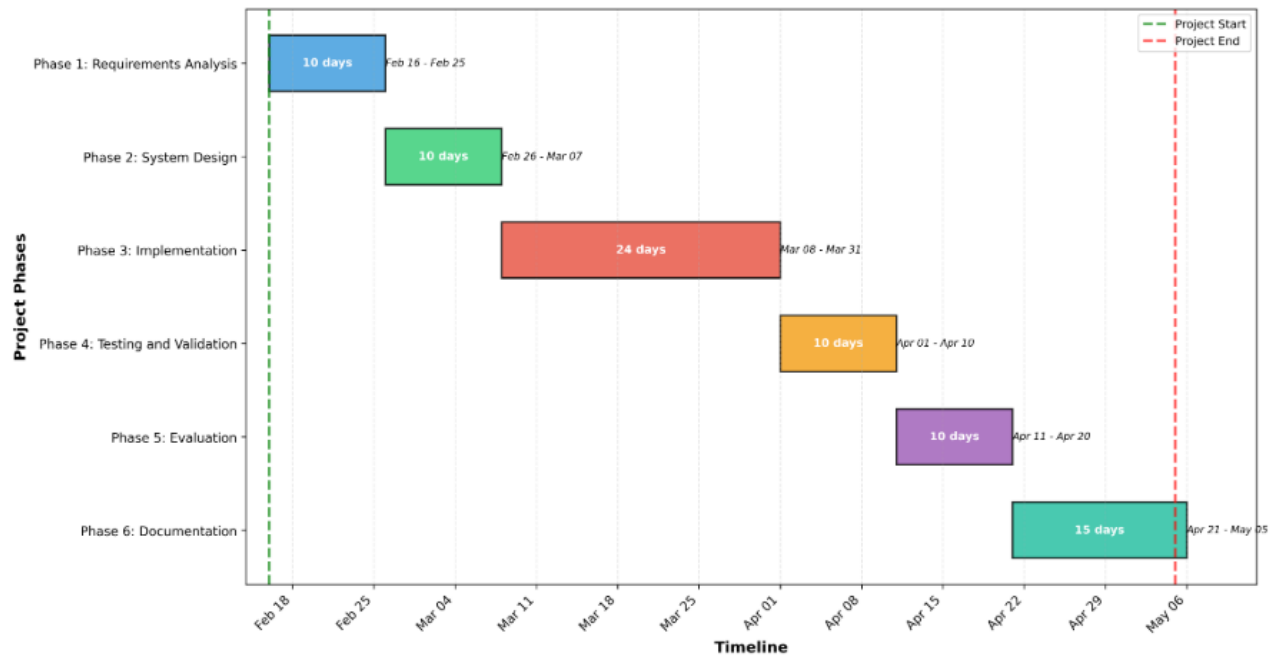


Fig 2: Gantt Chart for the dissertation

8.3 Milestones and Deliverables

Milestone	Date	Phase	Deliverables
M1: Requirements Complete	Feb 25, 2026	Phase 1	- Literature review document. - System requirements specification. - Preprocessed HDFS and BGL datasets. - GitHub repository initialized. - Development environment configured
M2: Design Complete	Mar 07, 2026	Phase 2	- System architecture diagram. - Component interface specifications. - Knowledge base schema. Baseline system designs. - Pseudo-code for agentic controller

M3: Implementation Complete	Mar 31, 2026	Phase 3	• Functional Log Preprocessor. • All baseline systems (Rule-based, Isolation Forest, Non-agentic RAG). • Complete Agentic RAG framework. • Integration test results.
M4: Validation Complete	Apr 10, 2026	Phase 4	• Unit test results. • Integration test report. • Validated preprocessing pipeline. • Pilot experiment results.
M5: Evaluation Complete	Apr 20, 2026	Phase 5	• Experimental results on HDFS and BGL datasets. • Performance comparison tables. • Statistical analysis report. • Visualization figures.
M6: Final Submission	May 05, 2026	Phase 6	• Complete dissertation document. • All chapters finalized. • Bibliography formatted. • Final submission-ready PDF.

8.4 Risk Management

Risk	Likelihood	Impact	Mitigation
Dataset download issues	Low	Medium	• Pre-download from multiple sources. • Use cached versions.
LLM computational limits	Medium	Medium	• Use smaller quantised models (4-bit). • Access device with dedicated GPUs.
Baseline underperformance	Medium	High	• Document all hyperparameters.

			Follow published configurations.
Timeline delays	Medium	High	- Build buffer time. - Work on parallel tasks. - Weekly progress reviews.
Negative results	Low	Low	- Frame as valid scientific finding. - Focus on analysis of why.

8.5 Ethical Considerations

The ethical considerations that comes with this dissertation and the development of the project is dynamic. One of which is primarily related to **Data Privacy**. To tackle that consideration, this dissertation will be using the HDFS and BGL datasets, which are public and anonymised with no personal information. Another ethical consideration is to deliver a responsible AI. The alerts that will be generated from the developed system will include confidence scores and source attribution for human validation. This mitigates the ethical consideration for the responsible AI.

References

- Arrieta, A.B., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., Chatila, R. and Herrera, F. (2020) Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58, pp.82-115.
- Blair, G. et al. (2023) Retrieval-augmented generation for legal case analysis. In: *Proceedings of the Natural Legal Language Processing Workshop*, pp.45-58.
- Chen, M. et al. (2021) Empirical study of log analysis challenges in distributed systems. *ACM Transactions on Software Engineering and Methodology*, 30(4), pp.1-35.
- Chen, S., Wong, W. and Zhang, L. (2023) Evaluating explainability in anomaly detection systems: A human-centred approach. *IEEE Transactions on Software Engineering*, 49(6), pp.3421-3439.
- Du, M., Li, F., Zheng, G. and Srikumar, V. (2017) DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pp.1285-1298.
- Es, S., James, J., Espinosa-Anke, L. and Schockaert, S. (2023) RAGAS: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*.
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J. and Wang, H. (2023) Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.
- Gartner (2023) *The Cost of Downtime*. Available at: <https://www.gartner.com> (Accessed: 12 January 2026).
- Guo, H., Yuan, S. and Wu, X. (2021) LogBERT: Log anomaly detection via BERT. In: *2021 International Joint Conference on Neural Networks (IJCNN)*, pp.1-8.
- He, P., Zhu, J., Zheng, Z. and Lyu, M.R. (2017) Drain: An online log parsing approach with fixed depth tree. In: *2017 IEEE International Conference on Web Services (ICWS)*, pp.33-40.
- He, S., He, P., Chen, Z., Yang, T., Su, Y. and Lyu, M.R. (2020) A survey on automated log analysis for reliability engineering. *ACM Computing Surveys*, 54(6), pp.1-37.

Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) Design science in information systems research. *MIS Quarterly*, 28(1), pp.75-105.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A. and Fung, P. (2023) Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), pp.1-38.

Le, V.H. and Zhang, H. (2022) Log parsing evaluation in the era of modern software systems. In: *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pp.1574-1586.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. and Kiela, D. (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. In: *Advances in Neural Information Processing Systems (NeurIPS)*, 33, pp.9459-9474.

Liu, F.T., Ting, K.M. and Zhou, Z.H. (2008) Isolation forest. In: *2008 Eighth IEEE International Conference on Data Mining*, pp.413-422.

Meng, W., Liu, Y., Zhu, Y., Zhang, S., Pei, D., Liu, Y., Chen, Y., Zhang, R., Tao, S., Sun, P. and Xu, H. (2019) LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In: *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, pp.4739-4745.

Oliner, A. and Stearley, J. (2007) What supercomputers say: A study of five system logs. In: *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp.575-584.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N. and Scialom, T. (2023) Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*.

Xu, W., Huang, L., Fox, A., Patterson, D. and Jordan, M.I. (2009) Detecting large-scale system problems by mining console logs. In: *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*, pp.117-132.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y. (2023) ReAct: Synergizing reasoning and acting in language models. In: *International Conference on Learning Representations (ICLR)*.

Zhang, Y. et al. (2024) Retrieval-augmented generation for clinical decision support: A systematic evaluation. *Journal of Biomedical Informatics*, 145, 104456.